

Large language models

[attachment:90405e97-c3a1-48c9-a7b0-7b76e1a8a351:06_Large_language_models.pdf](#)

Final short study summary

- LLMs are next-token predictors trained on huge corpora, typically using transformer architectures.
- Three common architectures:
 - **Encoder** (bidirectional) for understanding tasks like classification and NER.
 - **Decoder** (causal) for text generation.
 - **Encoder-Decoder** for tasks like translation and summarization.
- Training pipeline usually includes preprocessing, pretraining, instruction tuning, and alignment (e.g., RLHF).
- Transformers outperform RNNs by using self-attention to model long-range dependencies in parallel.
- Self-attention uses **Q/K/V** projections and computes $\text{softmax}(QK^{\text{top}/\sqrt{d_k}})V$.
- **Multi-head attention** runs several smaller attentions in parallel to capture different relationships.
- **FFN layers** apply non-linear transformations per token, while residual connections + layer norm stabilize deep training.
- **Masking** includes padding masks and causal (look-ahead) masks for decoders.
- **Positional encodings** inject order information (absolute sinusoidal/learned or relative biases like T5 and ALiBi).
- Decoding strategies (greedy, temperature, top-k, top-p) trade off determinism vs. diversity at inference.

Text Mining — Finals Study Notes

Large Language Models & Transformers

Based on lecture slides & recordings

1. What Are Large Language Models (LLMs)?

LLMs are computational systems trained to predict the next token in a sequence. They use large context windows and billions of parameters to perform complex tasks.

Key characteristics:

- Predict the next token (word/subword) in a sequence
- Use billions of parameters and large context windows
- Built on transformer neural networks
- Capable of: reasoning, coding, translation, creative writing, summarization, Q&A

💡 LLMs generalize from enormous amounts of text data — no manual feature engineering needed.

2. LLM Architectures — The Three Types

There are three main transformer-based architectures, each suited to different tasks:

Architecture	Nickname	Direction	Best for	Examples
Encoder	"Readers"	Bidirectional	Classification, NER, sentiment analysis	BERT, RoBERTa, ALBERT
Decoder	"Writers"	Left-to-right only	Text generation, language modeling	GPT, LLaMA, Claude, Mistral, Gemini
Encoder-Decoder	"Translators"	Both	Translation, summarization	T5, BART, Pegasus

Encoder ("Readers")

- Transforms input sequence into vector representations
- Uses information from both left AND right (bidirectional)
- Trained with Masked Language Modeling (MLM) — hide tokens, predict them
- Used for: classification, sentiment analysis, Named Entity Recognition (NER)

Decoder ("Writers")

- Generates tokens one at a time, left-to-right
- Most dominant architecture today
- With enough data and parameters, no encoder needed
- Simpler — no cross-attention between two parts

💡 Decoders use a look-ahead (causal) mask so the model cannot see future tokens during training.

Encoder-Decoder ("Translators")

- Encoder reads and understands the input sequence
- Decoder generates output based on encoder's representations
- Connected through cross-attention
- Queries come from decoder, Keys and Values come from encoder

3. LLM Training Pipeline

Training happens in four main phases:

Phase 1: Preprocessing

- Data cleaning — remove noise, duplicates, low quality text
- Tokenization — split text into tokens (subwords, characters, words)

Phase 2: Pretraining

- Trained on massive text corpora
- Task: predict the next token — called self-supervised learning
- Uses cross-entropy loss and backpropagation to learn weights

💡 Self-supervised = labels come from the data itself (next word is the label).
No human annotation needed.

Phase 3: Instruction Tuning (Fine-tuning)

- Uses a task-specific dataset (QA, coding, summarization, translation)
- Same cross-entropy + backpropagation, but updates weights toward specific tasks

Phase 4: Alignment

- Final phase — makes the model behave helpfully and safely
- Uses Reinforcement Learning from Human Feedback (RLHF) or similar
- Humans provide accepted/rejected response pairs
- Model learns to produce outputs more aligned with human preferences

4. LLM Evaluation

- **Perplexity:** measures quality of token probability prediction. Lower = better, but not sufficient alone.
- **Benchmarks:** e.g., MMLU (Massive Multitask Language Understanding) — tests many domains.
- **Multiple aspects evaluated:** toxicity, ethics, energy usage, privacy, bias, fairness.

💡 Because LLMs are used for many different tasks, evaluation is inherently multi-dimensional — no single metric captures everything.

5. The Transformer Architecture

Origin

- Introduced in Vaswani et al. 2017 — "Attention Is All You Need"
- Originally encoder-decoder for machine translation
- Key innovation: self-attention replaces recurrent connections (RNNs/LSTMs)

Why Transformers Beat RNNs

Feature	RNNs	Transformers
Token access	Through hidden state (sequential)	Direct via attention (parallel)
Long-range dependencies	Hard (vanishing gradient)	Easy (direct attention)
Training speed	Sequential — slow	Parallelized — fast
Contextual embeddings	Partial	Rich (self-attention on full sequence)

Key Properties of Self-Attention

- Each token has direct access to every other token in the context
- Creates contextual embeddings — the same word gets different representations based on context
- Enables parallel computation across all positions

6. Transformer Block — The Core Building Block

Each transformer block processes a token stream and contains:

- **Self-Attention:** lets each token attend to all others (or past tokens for decoders)
- **Feedforward Layer (FFN):** processes and transforms the information gathered by attention
- **Residual Connections:** adds the block's input directly to its output (prevents vanishing gradients, allows skipping)
- **Layer Normalization:** keeps activations on a stable scale for better training

Block Formula (Pre-Norm variant — modern):

```

t1 = LayerNorm(xi)
t2 = MultiHeadAttention(t1, [t11...t1n])
t3 = t2 + xi (residual)
t4 = LayerNorm(t3)
t5 = FFN(t4)
hi = t5 + t3 (residual)

```

Blocks are stacked (12–100+ layers). Each block outputs the same dimensionality as its input — this is what allows stacking.

Parallelized block computation (matrix form):

```
O = X + MultiHeadAttention(LayerNorm(X))
H = O + FFN(LayerNorm(O))
```

7. Attention Mechanism — Explained Step by Step

Core Idea

Each token (query) looks at all other tokens (keys) to decide how much attention to pay them, then sums the values weighted by those attention scores.

The Three Vectors

Vector	Role	Source Matrix
Query (q)	What this token is looking for	WQ
Key (k)	What information this token offers	WK
Value (v)	The actual content to be summed	WV

Scaled Dot-Product Attention Formula

```
Attention(Q, K, V) = softmax( QKT / √dk ) · V
```

- Q, K, V are matrices computed from input X via learned weight matrices WQ, WK, WV
- QK^T computes similarity scores between all query-key pairs ($n \times n$ matrix)
- Dividing by $\sqrt{dk}$ prevents extremely large dot products (keeps variance = 1)
- Softmax normalizes scores into probabilities (attention weights)
- Final result: weighted sum of value vectors

Why Scale by $\sqrt{dk}$?

- High-dimensional vectors → large dot product magnitudes
- Large values → softmax becomes near-deterministic (one token gets all attention = "hard" attention)

- Small gradients → slower learning
- Dividing by $\sqrt{d_k}$ keeps the variance stable

Multi-Head Attention

Instead of one large attention, multiple smaller attention heads run in parallel:

- Each head uses smaller projection matrices ($d_{\text{model}} = d_k \times N_{\text{heads}}$, e.g., $512 = 8 \times 64$)
- Each head focuses on a different aspect of the token relationships
- Outputs of all heads are concatenated and projected back: $a_i = [\text{head}_1; \text{head}_2; \dots \text{head}_A] \cdot W_O$
- **Benefits:** more efficient computations, softer attention (better distribution), acts like an ensemble

8. Feedforward Layer (FFN)

The FFN is where actual "thinking" happens — attention only gathers context, FFN transforms it:

$$\text{FFN}(x_i) = g(x_i W_1 + b_1) W_2 + b_2$$

Component	Role
W_1 (expansion matrix)	Expands to $4 \times d_{\text{model}}$ dimensions — more room for computation
g (activation: ReLU/GELU)	Introduces non-linearity
W_2 (contraction matrix)	Projects back to d_{model} — condenses and integrates features
Shared weights per layer	Same W_1, W_2 for all positions in a layer (enables parallelization)

9. Layer Normalization

Normalizes activations using the z-score formula:

$$\text{LayerNorm}(x) = \gamma \cdot (x - \mu) / \sigma + \beta$$

- **γ (scale) and β (shift):** learnable parameters — allow the model to undo normalization if needed
- **Pre-norm (modern):** normalize BEFORE attention/FFN → more stable training, handles larger learning rates
- **Post-norm (original):** normalize AFTER — gradient can vanish before reaching attention/FFN

10. Masking

Masking is added to the attention score matrix to block certain positions. The formula becomes:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k} + M) \cdot V$$

Mask type	Purpose	Where $-\infty$ appears
Padding mask	Ignore padding tokens (sequences padded to equal length)	Columns corresponding to padding positions
Look-ahead mask (causal)	Decoder: prevent seeing future tokens during training	Upper triangle of QK^T matrix

💡 During decoder training, teacher forcing is used — correct tokens are fed as input (not the model's own predictions), enabling full parallelization despite the sequential nature of generation.

11. Positional Encodings

Transformers process all tokens in parallel — without positional information they'd treat input as a bag of words.

Absolute Positions

- **Simple integer:** add 1, 2, 3... — not bounded, later positions outweigh embeddings
- **Learned positions (BERT):** each position has a learned embedding — later positions trained on less data
- **Fixed sinusoidal (original transformer):** uses sin/cos functions at different frequencies

$$\begin{aligned} PE(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}) \\ PE(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_{\text{model}})}) \end{aligned}$$

- Values bounded between -1 and 1
- Every position is unique
- Relative distances remain consistent regardless of absolute position

Relative Positions

Instead of absolute position, a bias based on token distance is added to the attention score:

$$QK^T / \sqrt{d_k} + B(i-j)$$

- **T5**: learnable biases with bucketing (groups of distances share a bias)
- **ALiBi (BLOOM)**: fixed linear bias: $B = -m \cdot |i-j|$, different slope m per attention head

12. Token Generation Strategies (Decoding)

Strategy	How it works	When to use
Greedy decoding	Always pick the highest-probability token	Math, factual Q&A — deterministic output needed
Temperature sampling	Divide logits by T before softmax. $T < 1$: sharper; $T > 1$: flatter (more random)	Creative tasks — $T \approx 0$ is like greedy
Top-k sampling	Only sample from top k most likely tokens	Prevents rare/weird words
Top-p (nucleus)	Sample from smallest set of tokens with cumulative probability $\geq p$	Dynamic k — more flexible than fixed top-k

💡 These strategies can be combined: e.g., temperature + top-p is common in practice.

13. Decoder: Training vs. Inference

Training

- Task: predict the next token
- Teacher forcing: correct tokens fed as input (not model's own output) → allows parallel computation
- Look-ahead mask prevents seeing future tokens
- Loss computed at every position and averaged

Inference (Autoregressive generation)

- No teacher — model uses its own previous output
- Sequential loop: generate token → append to sequence → process again
- KV-caching: store Key and Value matrices for past tokens to avoid recomputation; only compute Q for new token
- Prompt processing (prefill): entire prompt processed in parallel; last token output used to start generation
- Few-shot learning: labeled examples in the prompt → in-context learning (no weight updates)

14. Encoder Training

Masked Language Modeling (MLM) — used in BERT

- Select 15% of tokens randomly
- Of those: 80% replaced with [MASK], 10% replaced with random word, 10% kept original
- Model must predict the original token — learns contextual relationships

💡 The mix of mask/random/original teaches: learn from context (masked), detect errors (random), maintain quality for unmasked (original).

Auxiliary Tasks

- **Next Sentence Prediction (NSP)**: classify if sentence B follows sentence A — considered too easy, often removed
- **Sentence Order Prediction (SOP)**: predict whether sentences are in correct order — harder and more useful

ELECTRA-Style Training (Replaced Token Detection)

- Generator (small encoder): fills in masked tokens (standard MLM)

- Discriminator (main encoder): decides if each token is real or replaced (binary classification)
- **Advantage:** uses ALL tokens (not just 15%) → much more data-efficient than BERT
- **No [MASK] mismatch:** discriminator never sees [MASK] in training, so no train/inference discrepancy

15. Model Outputs: Heads

Residual Stream Output

- After all transformer blocks, each token has a contextual embedding
- Same dimensionality as input — can be used for semantic search, similarity, or downstream tasks

Language Modeling Head (Decoders)

- Linear layer (often weight-tied with embedding matrix): transforms d_{model} → vocabulary size
- Softmax layer: converts to probability distribution over tokens

Encoder Task Heads

- **Token classification head:** one label per token (e.g., NER)
- **Sequence classification head:** one label for whole sequence (e.g., sentiment)
- Usually uses the [CLS] token (artificially prepended) — accumulates full-sequence info
- Multiple heads can be trained simultaneously on different tasks
- **Auxiliary heads:** intermediate heads added to improve gradient flow; not used at inference time

Quick Reference — Key Terms & Formulas

Term	Definition
Token	Smallest unit of text (subword, word, character)
Self-attention	Each token attends to all others in the sequence

Term	Definition
Multi-head attention	Multiple parallel attention heads, each focusing on different aspects
Scaled dot-product attention	$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$
Teacher forcing	Feed correct previous token during training (not the model's prediction)
KV caching	Store K, V from past tokens to speed up inference
Positional encoding	Adds position information since transformers process in parallel
Look-ahead mask	Sets future positions to $-\infty$ in attention so decoder can't cheat
Perplexity	How surprised the model is by the test data (lower = better)
MLM	Masked Language Modeling — BERT's pretraining task
RLHF	Reinforcement Learning from Human Feedback — used in alignment
In-context learning	Using examples in the prompt to improve output (no weight updates)
CLS token	Special token prepended to sequences; used for sequence classification
Weight tying	Sharing embedding and unembedding matrix weights
ALiBi	Attention with Linear Biases — relative position encoding for decoders
Cross-attention	Decoder attends to encoder output (Q from decoder, K/V from encoder)

Good luck on your finals! 🎓